ESPE

UNIVERSIDAD DE LAS FUERZAS ARMADAS

INNOVACIÓN PARA LA EXCELENCIA

DEPARTAMENTO DE CIENCIA DE LA COMPUTACION

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN (ITIN)

TEMA: AUTOEVALUACIÓN CAPITULO 1

INTEGRANTES:
LEOPOLDO BOLAÑOS
MEDINA MATIAS
MARTIN PALACIOS

ASIGNATURA: METOD DE DESARROLLO SOFTWARE

NRC: 30693

FECHA: 13/04/2026

DOCENTE: ING.JENNY ALEXANDRA RUIZ ROBALINO

FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: **Ciencias de la Computación**
	Carrera: **ITIN** Asignatura: **Metodología de Desarrollo de Software** NRC: **30693**
	Apellidos y nombres del estudiante: **G7**

ANTECEDENTES

1. La ingeniería de software se ha desarrollado como una disciplina fundamental dentro de la informática, con el objetivo de mejorar la calidad y eficiencia en la creación de sistemas, respondiendo a problemas históricos como la crisis del software.
2. A lo largo del tiempo, han surgido diversas metodologías, paradigmas y modelos de desarrollo que permiten estructurar el proceso de creación de software, adaptándose a las necesidades del entorno tecnológico actual.

OBJETIVO

1. Analizar los conceptos fundamentales de la ingeniería de software, mediante la revisión de sus definiciones, actividades, costos, retos y atributos, para comprender su importancia en el desarrollo de sistemas eficientes y de calidad.
2. Comparar los diferentes enfoques, métodos y paradigmas de desarrollo de software, a través del estudio de modelos como el incremental, metodologías ágiles y tradicionales, para identificar sus características y su aplicación en problemas reales.

DESARROLLO

1. **¿Qué es ingeniería de software?**

Para mí, la ingeniería de software es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento de software (Potts, 1993). Consiste en utilizar el

conocimiento científico de manera práctica, estableciendo principios de la ingeniería para construir, de forma económica, un software que sea confiable y trabaje con eficiencia en máquinas reales (Bauer, 1968).

2. ¿Cuáles son las actividades fundamentales de la ingeniería de software?

Todo proceso de software, sin importar el modelo a elegir, debe incluir cuatro actividades básicas requeridas por nuestra disciplina: especificación, diseño e implementación, validación y evolución (Somerville, 2011). La especificación define la funcionalidad y las restricciones operativas; El diseño e implementación se encarga de desarrollar el código para cumplir con dichas especificaciones; La validación examina el software para asegurar el cumplimiento de los requerimientos exigidos por el cliente, y la evolución le da al software la flexibilidad de transformarse frente a nuevas necesidades (Somerville, 2011).

3. ¿Cuál es la diferencia entre ingeniería de software y ciencias de la computación?

La principal diferencia radica en su enfoque práctico frente a lo teórico. En las ciencias de la computación, el software es simplemente el lenguaje con el que expresamos ideas y exploramos una amplia gama de problemas de computación teóricos y de hardware (ACM, 2020). En cambio, para nosotros en la ingeniería de software, el énfasis está directamente puesto en la creación sistemática de aplicaciones a gran escala que satisfagan las necesidades robustas y del mundo real.

4. ¿Qué es software?

Un producto software es el resultado final de un proceso de desarrollo o ciclo de vida, donde las actividades están debidamente estructuradas y secuenciadas para cumplir con los requisitos funcionales y no funcionales solicitados por la parte interesada (Somerville, 2011). Los elementos que componen al software no son solo los algoritmos de código, sino los programas, procedimientos, la documentación y los datos relacionados a este.

5. **¿Cuáles son los atributos del buen software?**

Un buen producto de software debe brindar la funcionalidad y el rendimiento requerido por el usuario, destacando por ser mantenible, confiable y fácil de utiliza. Según el modelo de calidad ISO/IEC 25010, podemos evaluarlo mediante ocho características principales: adecuación funcional, fiabilidad, eficiencia de desempeño, usabilidad, seguridad, compatibilidad, mantenibilidad y portabilidad.

6. **¿Cuáles son los principales retos que enfrenta la ingeniería de software?**

Históricamente, el principal reto fue la "crisis del software", referida a los graves problemas que se generaban debido a la mínima eficacia de las empresas al momento de realizar proyectos y la baja calidad resultante. Actualmente y hacia el futuro, el desafío se centra en lograr una mayor integración entre la ingeniería del software y la de sistemas, destacando como prioridad absoluta el cumplimiento de los requisitos no funcionales y, sobre todo, de la seguridad (Oktaba, 2018).

7. **¿Cuáles son los costos de la ingeniería de software?**

Sorprendentemente, el mayor impacto económico se da en la fase de mantenimiento, ya que esta etapa puede representar más del 60% de los costos totales de desarrollo de un sistema. Además, los costos se incrementan de forma muy agresiva cuando intentamos agregar funcionalidad a un sistema que ya está en operación si las prácticas de desarrollo iniciales fueron deficientes.

8. **¿Cuáles son los mejores métodos y técnicas de la ingeniería de software?**

Generalmente se clasifican en metodologías estructuradas y orientadas a objetos. Hoy en día, nuestra mejor opción son las metodologías ágiles (como XP, SCRUM y AUP), ya que se combinan para realizar entregas rápidas del software en incrementos, minimizando la documentación burocrática y concentrando todo nuestro esfuerzo en el código en sí (Somerville, 2011). Scrum, particularmente, se ha consolidado como el estándar de trabajo en la industria (Solis, 2024).

9. **¿Qué entiende por paradigmas de la ingeniería de software?**

Los paradigmas son simplemente los modelos generales de proceso que nos dictan la arquitectura y las estrategias para desarrollar el sistema. Entre los más conocidos tenemos el clásico modelo en cascada, donde las fases de planificación, diseño y pruebas están separadas secuencialmente, el modelo de desarrollo incremental, que entrega versiones operativas progresivas, y la ingeniería orientada a la reutilización de componentes.

10. **¿Cuál es la definición del modelo incremental versus otros paradigmas? Características y diferencias con otras ingenierías.**

El modelo incremental vincula nuestras actividades para construir y entregar una serie de versiones (incrementos), donde cada versión le añade nuevas funcionalidades a la anterior, lo que ayuda a que el cliente entienda de mejor manera el resultado final mientras avanza el proyecto. Su diferencia con otras ingenierías tradicionales es la naturaleza de nuestro producto: el software es un elemento netamente lógico que se desarrolla o modifica con el intelecto, no se "fabrica" en el sentido clásico. Asimismo, nuestro software no sufre un desgaste físico tradicional por fricción, sino que se "desgasta" con el transcurso del tiempo frente a su entorno.

CONCLUSIONES

1. La ingeniería de software es una disciplina esencial que permite desarrollar sistemas confiables mediante procesos estructurados.
2. Las actividades fundamentales (especificación, diseño, validación y evolución) garantizan un desarrollo completo y ordenado del software.

RECOMENDACIONES

1. Aplicar metodologías de desarrollo adecuadas según el tipo de proyecto, priorizando enfoques ágiles cuando sea necesario.

2. Considerar desde el inicio los requisitos funcionales y no funcionales, especialmente la seguridad y el mantenimiento.

BIBLIOGRAFÍA

Bauer, F. L. (1968). *Software Engineering. [cite_start]Germany: Nato Science Committee[cite: 792].

López Echeverry, A. M., & Valencia Ayala, L. E. (2008). Introducción a la calidad de software. [cite_start]Dialnet[cite: 801].

Oktaba, H. (2018). 50 años de la ingeniería de software problemas, logros, tendencias y retos. [cite_start]*SG#58 tejiendo nuestra red*[cite: 803].

* Potts, C. (1993). Software-Engineering Research Revisited. [cite_start]*IEEE Software*[cite: 806].

* Solis, C. (2024). *Scrum esencial*. [cite_start]In Learning[cite: 810].

* Somerville, I. (2011). [cite_start]*Software Engineering 9*[cite: 811].